

YASH PANDEY

12214802722

CSE - Shift - I (2nd Year - 2023)

MAIT ROHINI

DIGITAL LOGIC & COMPUTER DESIGN (DLCD)

(2023-24) SESSION - 3rd Sem

(Complete - DLCD - Lab File)

INDEX

Sr. No.	Name of the Experiment	Page No.	Date of Experiment	Date of Submission	Remarks
<u>1.</u>	Verification of Truth Table of Logic Gates	2.	31-8-23	14-9-23	8 14/9
<u>2.</u>	To Realize The Design of Half Adder and Full Adder	5.	14-9-23	5-10-23	8 5/10/23
<u>3.</u>	To Realize The Design of Half Subtractor & Full Subtractor	7.	5-10-23	12-10-23	8 12/10/23
<u>4.</u>	Design and Implementation of 4-Bit Adder / Subtractor	9.	12-10-23	19-10-23	8 19/10/23
<u>5.</u>	To Realize MUX & De-MUX Using NAND Gates Only	11.	19-10-23	16-11-23	8 16/11/23
<u>6.</u>	Design and Implementation of Encoder & Decoder	13.	16-11-23	23-11-23	
<u>7.</u>	To Realize Grey To Binary and Binary To Grey Converter	16.		30-11-23	
<u>8.</u>	Construction & Verification of 2-Bit Ripple Counter	19.		30-11-23	
<u>9.</u>	Assembly Code in GNU Sim 8085 for Implement Data Transfer Instruction	21.		7-12-23	
<u>10.</u>	Assembly Code in GNU Sim 8085 for Arithmetic Inst. on Values	22.		7-12-23	
<u>11.</u>	Assembly Code in GNU Sim 8085 for Add of Two 8-Bit Number	23.		7-12-23	
<u>12.</u>	Assembly Code in GNU Sim 8085 for Factorial of a Number.	24.		7-12-23	

Experiment - 1.

1. Aim $\rightarrow$ Verification of Truth Table of The Logic Gates.

2. Apparatus $\rightarrow$ Breadboard, IC's, LED's, +5V Power Supply & Connecting Wires.

3. Theory:

(i) NAND Gate $\rightarrow$ The NAND Gate represents the Complement of The AND operation. It's Name is an abbreviation of NOT-AND. The Graphic symbol of NAND is consist of AND Symbol with a Bubble as output. It denotes The Complement of The operation performed on The output performed of AND Gate.

(ii) NOR Gate $\rightarrow$ The NOR Gate represents The Complement of The OR operation. It's Name is an abbreviation of NOT. The Graphic symbol for The NOR Gate consist of an OR Symbol with a bubble as output which denotes that a Complement operation is performed as output of OR Gate.

★ NAND and NOR Gates are called as Universal Gates as all The other Gates can be Implemented using these Two Gates.

4. Procedure

- $\rightarrow$ Connections are made as per The Circuit Diagram.
- $\rightarrow$ Switch on The Power Supply.
- $\rightarrow$ Apply different combinations of Inputs and observe The outputs, Compare The outputs with Their Respective Truth Tables.

"ON NEXT PAGE"

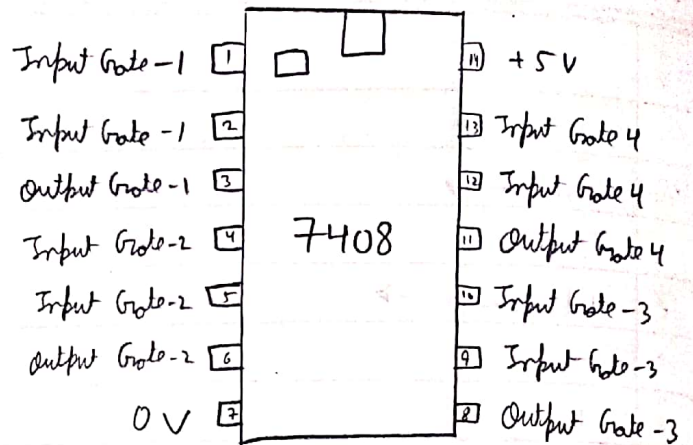
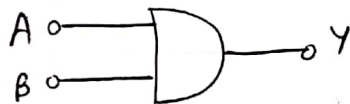
Teacher's Signature: _____

① AND Gate



$$Y = A \cdot B$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

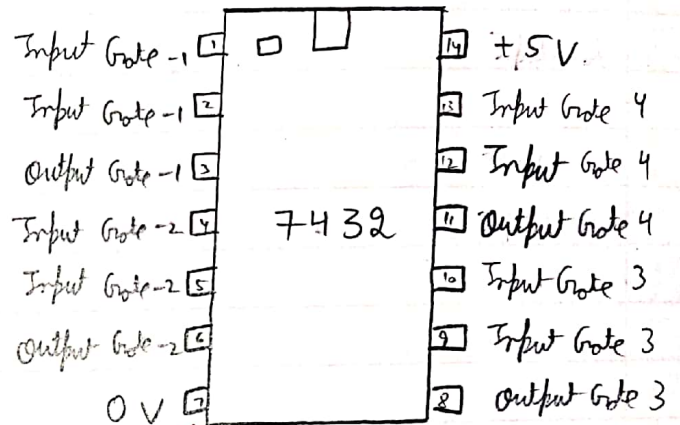
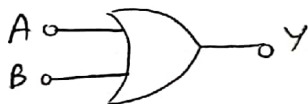


② OR Gate



$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

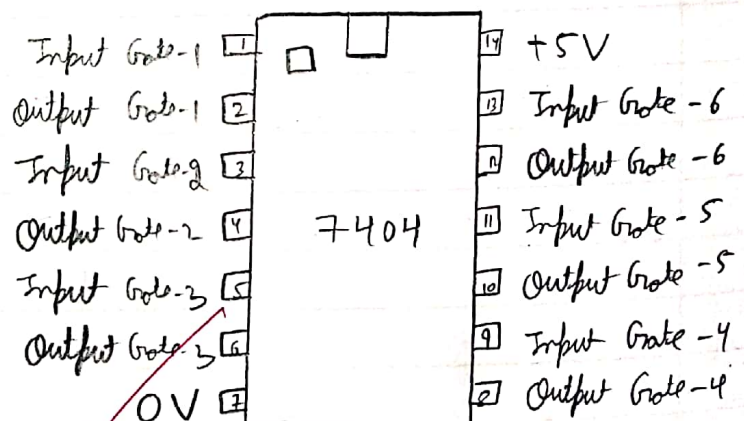
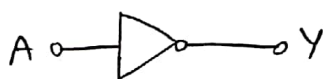


③ NOT Gate



$$Y = \bar{A}$$

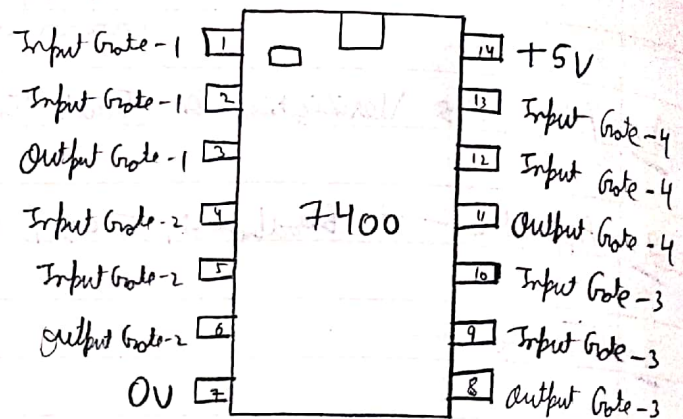
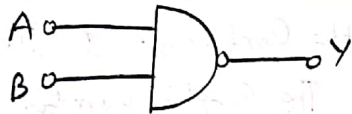
A	Y
0	1
1	0



④ NAND - Gate

→ $Y = \overline{A \cdot B} = \overline{A} + \overline{B}$

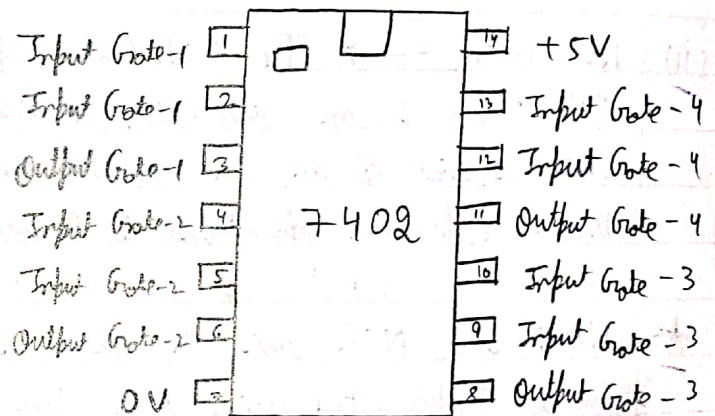
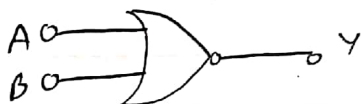
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



⑤ NOR - Gate

→ $Y = \overline{A + B} = \overline{A} \cdot \overline{B}$

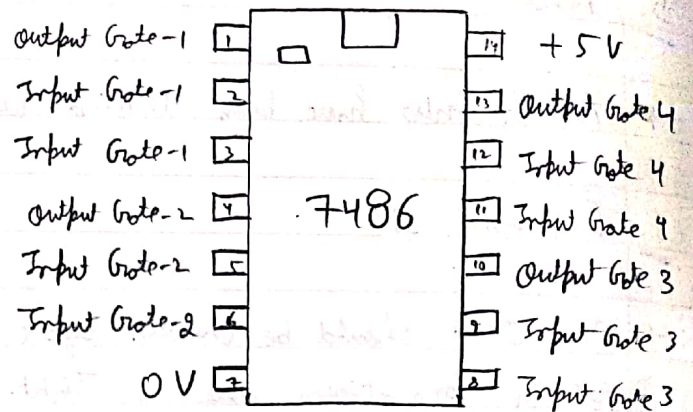
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0



⑥. X-OR Gate (EX-OR)

→ $Y = A \oplus B = AB + \bar{A}\bar{B}$

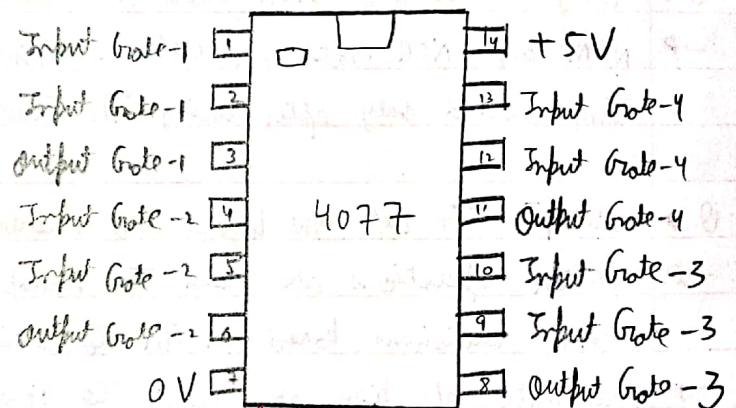
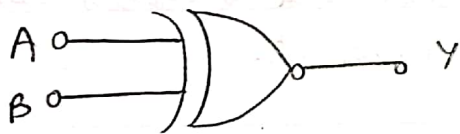
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0



⑦. X-NOR Gate (EX-NOR)

→ $Y = A \odot B = \bar{A}\bar{B} + AB$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1



5. Result

→ All The Gates have been Verified with Their Truth Tables.

6. Precautions

→ All IC's should be checked before Starting The Experiment.

→ All The Connections must be Tight.

→ Always Connect Ground first and Then Give Supply.

→ Switch Off The Power Supply after The Completion of The Experiment.

→ Viva - Questions.

Q → Why NAND and NOR Gates are Called as Universal Gates?

→ NAND and NOR Gates are Called as Universal Gates because all The Other Gates can be Implemented only after using The Two Gates.

Q → What is The Diff. b/w Logical and Arithmetic Operation?

→ Logical Operations are Used To Manipulate Boolean Values (True or False) and make decisions based on The Condition.

→ Arithmetic operation are used To Perform mathematical Calculations such as Addition, Subtraction, Multiplication and Division etc.

Q → What are Basic Gates?

→ Basic Gates are Basic Building Blocks for many Complex Operations.

→ Some Basic Gates in Digital Logic are AND, NOT and OR-Gate.

Teacher's Signature: _____

Experiment - 2.

1. Aim $\longrightarrow$ To Realize Half Adder and Full Adder using Basic Gates.
2. Apparatus Required $\longrightarrow$ Breadboard, IC-7486 (XOR), IC-7804 (AND), IC-7432 (OR), LEDs, 5V Battery Supply and Connecting Wires.
3. Theory
 - (i) Half Adder $\longrightarrow$ A Combinational Logic Circuit that Performs The Addition of 2 data Bits, A and B are called as Half Adder. Addition will Result in Two output bits as Sum and Carry as shown in figure. The Boolean Fⁿ which describes The Half Adder, $Sum = A \oplus B$
 $Carry = AB$
 - (ii) Full Adder $\longrightarrow$ The Half Adder doesn't Take the Carry from it's Previous Stage into account. This Carry Bit from The Previous Stage is called as Carry-in bit. A Combinational Logic Circuit which Adds two data Bits of 'A' and 'B' and Carry Bit Cin is called as full adder.

4. Procedure

- $\longrightarrow$ Verify The Gates.
- $\longrightarrow$ Make The Connections as per The Circuit Diagram.
- $\longrightarrow$ Switch on Vcc and apply Various Combination of Input from Truth Table.
- $\longrightarrow$ Note down The output Readings for The Half Adder and Full Adder.
- $\longrightarrow$ Sum and The Carry Bit for Diff. Circuit will Verify The Circuit

"ON NEXT PAGE"

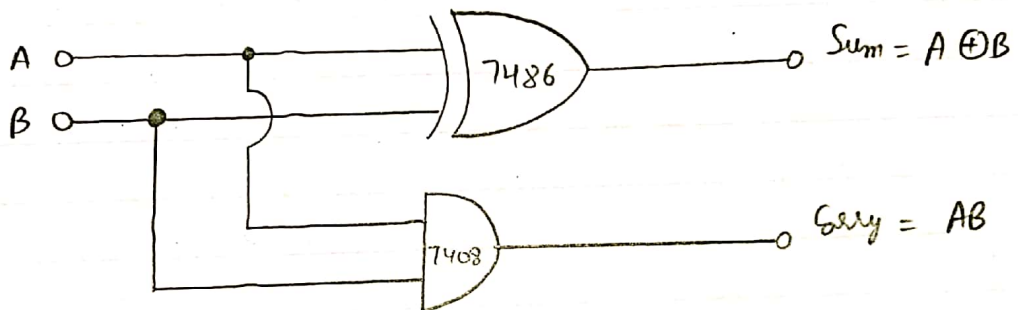
Teacher's Signature: _____

1. Half Adder (H.A)



Inputs		Outputs	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

*

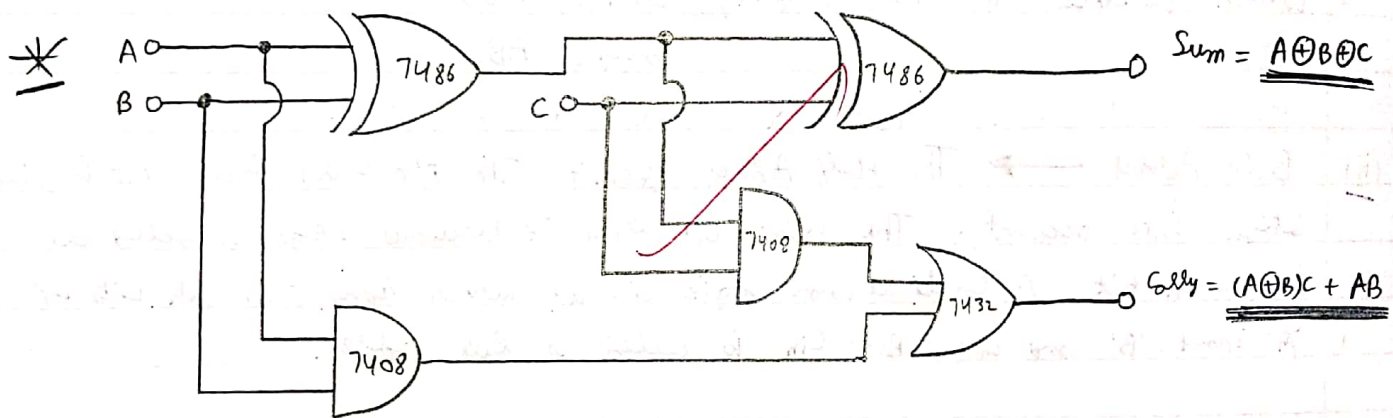


"HA Circuit Diagram"

2. Full Adder (F.A)



Inputs			Output	
A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



"FA Circuit Diagram"

5. Result

- Half Adder and Full Adder have been Realized Using The Basic Gates and Their Truth Table have been Verified.

6. Precautions

- All Ic's should be checked before the starting of the experiment.
→ All the connections must be tight.
→ Always connect the ground first and then give supply.
→ Switch off the power supply after the completion of the experiment.

→ Viva - Questions.

Q → Prove That The Sum of Full Adder is an XOR-Gate b/w its Inputs?

- To prove this, we can examine the Truth Table of a full Adder. The Sum output is '1' only when exactly one input is '1' and Behaviour is XOR-Gate.

Q → State The Difference b/w Half Adder and Full Adder?

- A Half Adder can only add two single bit binary numbers and produces a Sum & Carry output. However, a full adder can have 3 - single bit binary N.O. Taking into account an carry input from previous addition.

Q → Can We Design a Full Adder with using Half Adder?

- Yes, we can design Full Adder using 2 Half Adder.

Q → What are Combinational Circuit?

- These are digital circuits that produce output based solely on the current input values.

Teacher's Signature: _____

Experiment - 3.

1. Aim → To Realize Half Subtractor and Full Subtractor.

2. Apparatus → Breadboard, IC-7486 (XOR), IC-7408 (AND), IC-7404 (NOT), IC-7432 (OR), LED, +5V Power Supply and Connecting Wires.

3. Theory

(i) Half Subtractor → Subtracting a Single Bit Binary Value B from another A. It produces a difference bit C and Borrow Bit Bout. This operation is called as Half Subtraction and The Circuit To Realize it is called as Half Subtractor. The Boolean Eqⁿ describing Half Subtraction are,
$$\text{Diff}(D) = A \oplus B$$
$$\text{Borrow}(B) = \bar{A}B$$

(ii) Full Subtractor → Subtracting Two Single Bit Binary Value B, Cin from a Single Bit Value A produces a difference D and Borrow Bout. This is been called as full Subtraction.
$$\text{Diff.}(D) = A \oplus B \oplus C$$
$$\text{Borrow}(B) = \bar{A}B + \bar{A}C + BC = (A \oplus B)C + \bar{A}B$$

4. Procedure

- Check all The Components for Their Working.
- Insert The Appropriate IC for Their IC Base.
- Make The Connections as shown in The Logic Diagram.
- Verify The Results and observe The Output.

5. Result

- Half Subtractor and Full Subtractor have been Studied and Their Truth Table have been Verified.

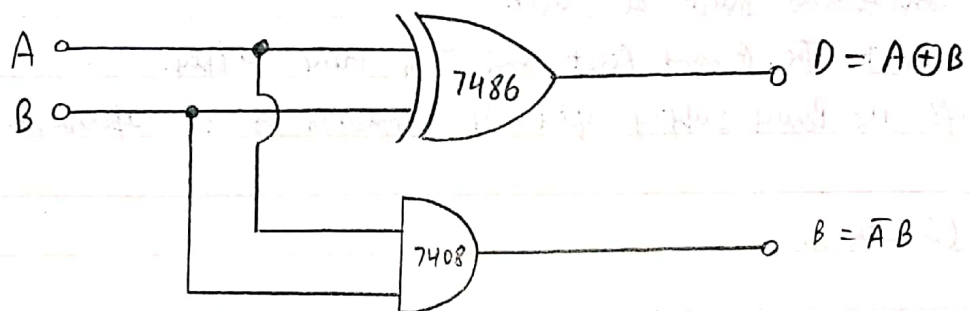
Teacher's Signature: _____

1. Half Subtractor (H.S)



Inputs		Outputs	
A	B	Diff-(D)	Borrow(B)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

*



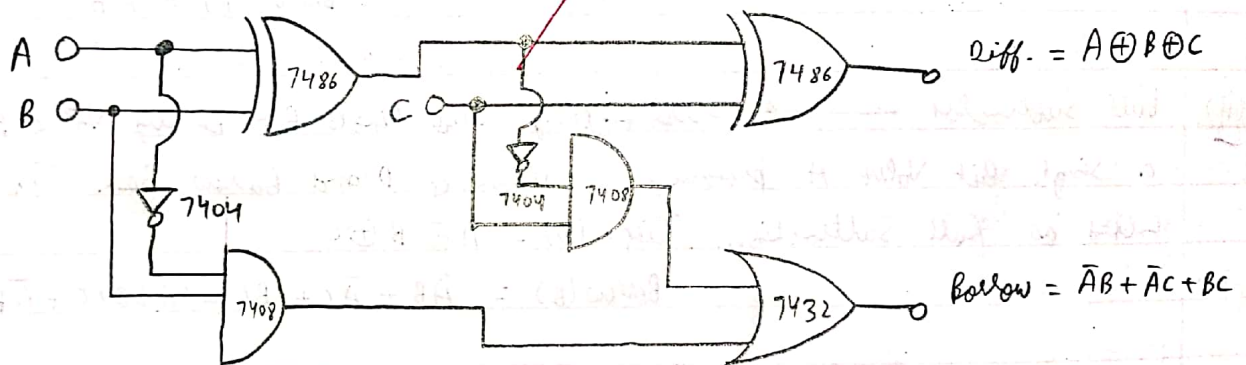
"H.S Circuit Diagram"

2. Full Subtractor (F.S)

→

Inputs			Outputs	
A	B	C	Diff (D)	Borrow (B)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

*



"F.S Circuit Diagram"

7. Precautions

- All The IC's must be checked before The Start of Experiment.
- All The Connections must be Tight.
- Always Connect The Ground first and Then Supply.
- Switch off The Power Supply after The Completion of Experiment.

Viva - Questions.

Q → What is The difference b/w Half Subtractor and Full Subtractor?

→ Half Subtractor

Full Subtractor

→ It Represents 2 Inputs.

→ It Represents 3 Inputs.

→ It Uses XOR, AND & OR Gate.

→ It Uses XOR, AND, OR and NOT Gate.

→ It Represents, $D = A \oplus B$
 $B = \bar{A}B$

→ It Represents, $D = A \oplus B \oplus C$
 $B = \bar{A}B + \bar{A}C + BC$

Q → List Down The IC's In Full Subtractor?

→ Full Subtractor is Consist of XOR (7486), NOT (7404), AND (7408) and OR - Gate (7432).

Q → Which method of Subtraction is Used in Computer?

→ In Computer, Subtraction is been Carry out by 2's Complement.

Teacher's Signature: _____

Experiment - 4.

1. Aim $\rightarrow$ Design and Implementation of 4-Bit Binary Adder / Subtractor.

2. Apparatus Required

S.NO	Components	Specification	Quantity
1.	4-Bit Binary Parallel Adder	IC - 74LS 83	1.
2.	XOR - Gate	IC - 7486	2

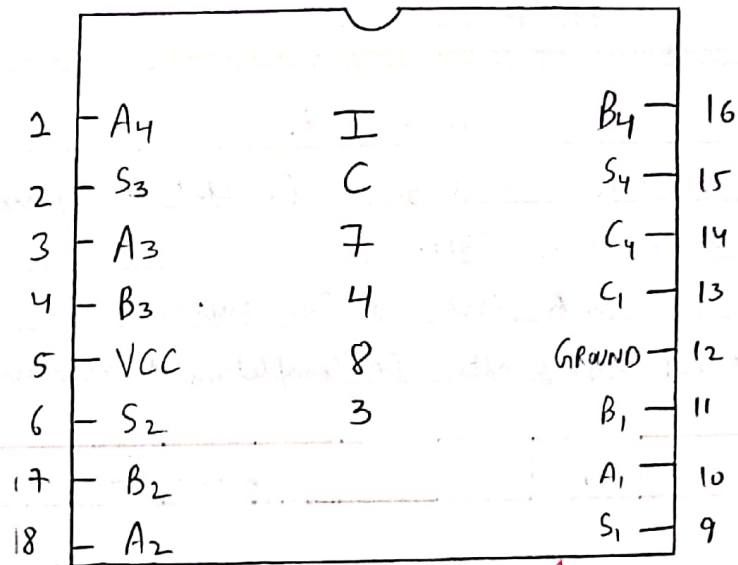
3. Theory $\rightarrow$ IC - Type 7483 is an 4-Bit Binary Parallel Adder. Each Input is Consist of 4-Bit. The Input Consist of A_4, A_3, A_2 and A_1 whereas The Input - B has B_3, B_2, B_1 and B_0 . It has four sum output which is Consist of S_4, S_3, S_2 and S_1 and C_0 which is Input Carry and C_4 is The output Carry. The VCC of Pin-5 & Ground at Pin-12.

(i) Adder $\rightarrow$ Binary Adder / Subtractor is a Special Type of Circuit that is used To perform both Operators. The Operation is Going To be used which depends on The Value Contained by The Control Signal. It is one of The Most Imp. Component To Work with Binary Adder / Subtractor. It is Required That The Knowledge of XOR - Gate, Binary Addition and Subtraction. The Control line determines whether The operators being performed in Either of Addition / Subtraction. This is done with help of $M_{10}(M)$.

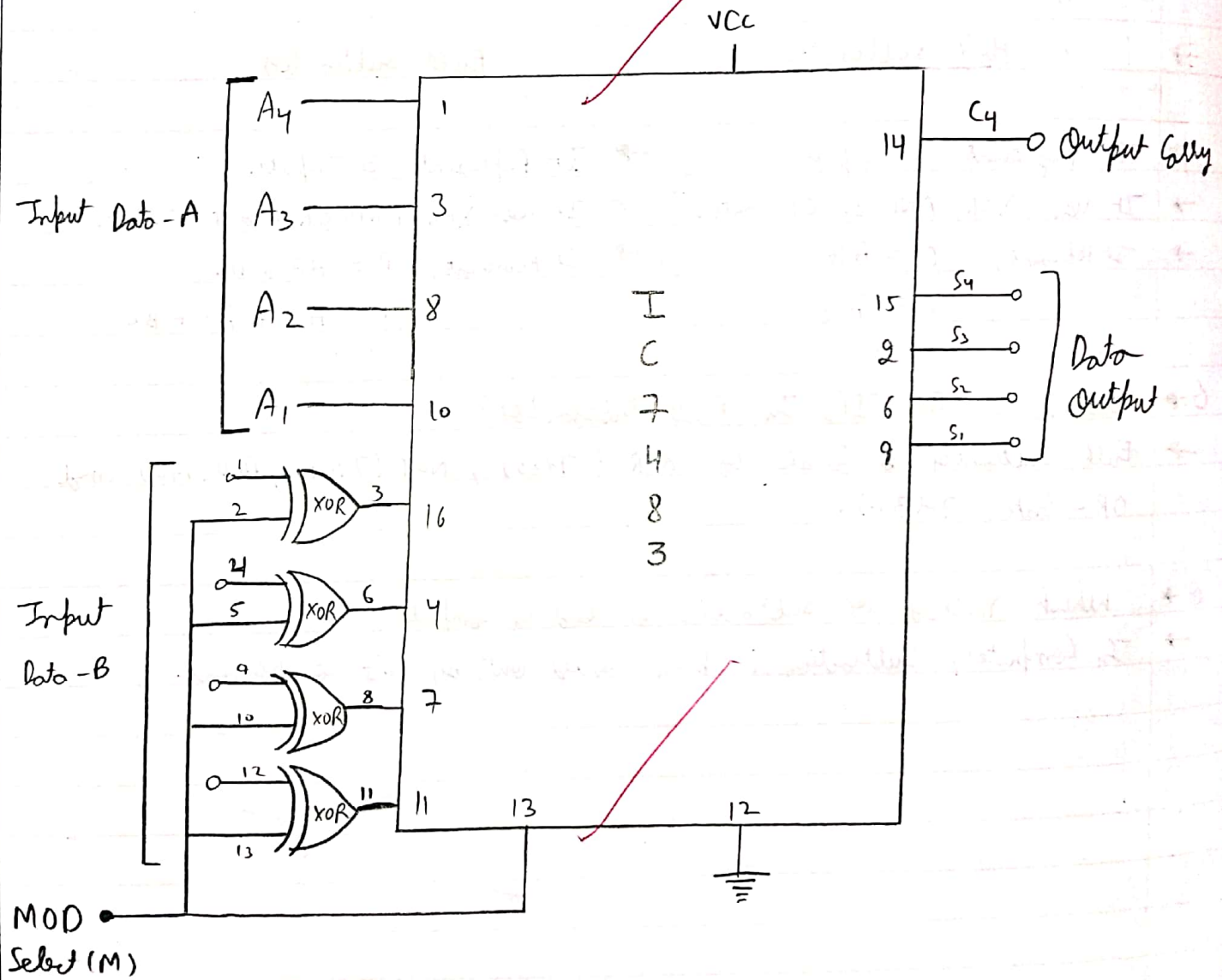
4. Procedure

- $\rightarrow$ Connect The Circuit as Shown in The Circuit Diagram.
- $\rightarrow$ Give The Input Values as specified in Truth Table.
- $\rightarrow$ Switch on The Power Supply.
- $\rightarrow$ Verify The outputs for Each Input Combination.
- $\rightarrow$ Repeat The Procedure with all Inputs of Truth Table.

Teacher's Signature: _____



"Pin - Diagram of IC - 7483"



"Truth Table of 4-Bit Adder / Subtractor"

M	Input - A				Input - B				Addition / Subtraction				C
	A ₁	A ₂	A ₃	A ₄	B ₁	B ₂	B ₃	B ₄	S ₁	S ₂	S ₃	S ₄	
0	1	0	0	1	0	0	0	1	1	0	1	0	0
0	1	0	0	1	0	1	1	0	1	1	1	1	0
0	1	1	0	1	0	1	0	1	1	0	1	0	1
0	0	0	1	1	1	1	0	0	1	1	1	1	0
1	1	0	0	1	0	0	0	1	1	0	0	0	0
1	1	0	0	1	0	1	1	0	0	0	1	1	1
1	1	1	0	1	0	1	0	1	1	0	0	0	0
1	1	0	1	1	1	0	0	0	0	0	1	1	0

5. Result $\longrightarrow$ Implementation of 4-Bit Adder / Subtractor is performed.

6. Precautions

- $\rightarrow$ Prevent Passage of High Current Through Circuit.
- $\rightarrow$ Make Proper Connections in The Breadboard.
- $\rightarrow$ Check The Condⁿ of LEO's before Use.

$\rightarrow$ Viva Questions.

Q $\rightarrow$ How many Inputs are in a Full Adder Circuit?

$\rightarrow$ There are 3 Inputs Provided in Full Adder as A, B, C, where, Sum is $(A \oplus B \oplus C)$ and Carry is $((A \oplus B)C + AB)$.

Q $\rightarrow$ What is The Truth Table of XOR?

$\rightarrow Y = A \oplus B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Q $\rightarrow$ What is The Condⁿ of Overflow in Case of Parallel Adder?

$\rightarrow$ When There is Carry (2) in C₄, Then, Overflow Condⁿ occurs.

Q $\rightarrow$ What is The Mod Selection bit in The Parallel Adder?

$\rightarrow$ Mod Selection Bit (M) $\rightarrow$ M=0 $\rightarrow$ 4-Bit Adder.

$\rightarrow$ M=1 $\rightarrow$ 4-Bit Subtractor.

Q $\rightarrow$ What is The Major Limitation in The Parallel Adder Circuit?

$\rightarrow$ Each Adder Has To Wait for Carry which is To be Generated from The Previous Adder in Chain.

Teacher's Signature: _____

Experiment - 5.~~~~~

1. Aim → To Realize Multiplexer and De-Multiplexer using NAND Gates.
2. Apparatus → Breadboard, IC 7400 (NAND), LED, 5V Power Supply and Connecting Batteries.
3. Theory → Multiplexers are Very useful Components in Digital System. They Transfer a large N.O. of Information over The Smallest N.O. of Channels under The Control of The Selection Signal. Multiplexer means one by one. A Multiplexer in The Circuit with many Input but only one output. By using The Select Line, we can select The Input for The output. Multiplexer is also called as data Selector because The Output Bit depends on The Input Data Bit that is been Selected.
→ De-Multiplexer performs The opposite Fⁿ of Multiplexer. They Transfer a Small N.O. of Information over a Very large N.O. of Channels under The Control of The Selecting Lines. The General De-Multiplexer Circuit has only one Input but 2^n Output signals.

4. Procedure

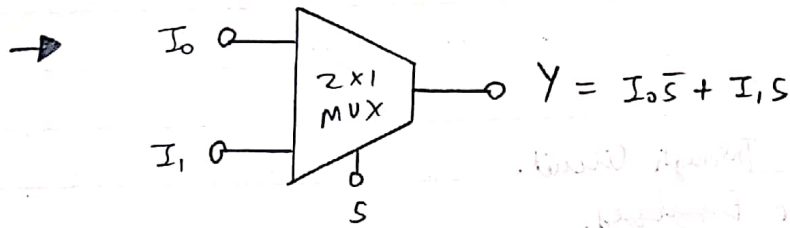
- Check all The Components before Working.
- Insert The Appropriate IC into The IC Base.
- Make Connections as shown in The Circuit.
- Verify The Result and observe The Inputs.

5. Result

- Multiplexer & De-Multiplexer have been studied and Their Truth Table have been Verified.

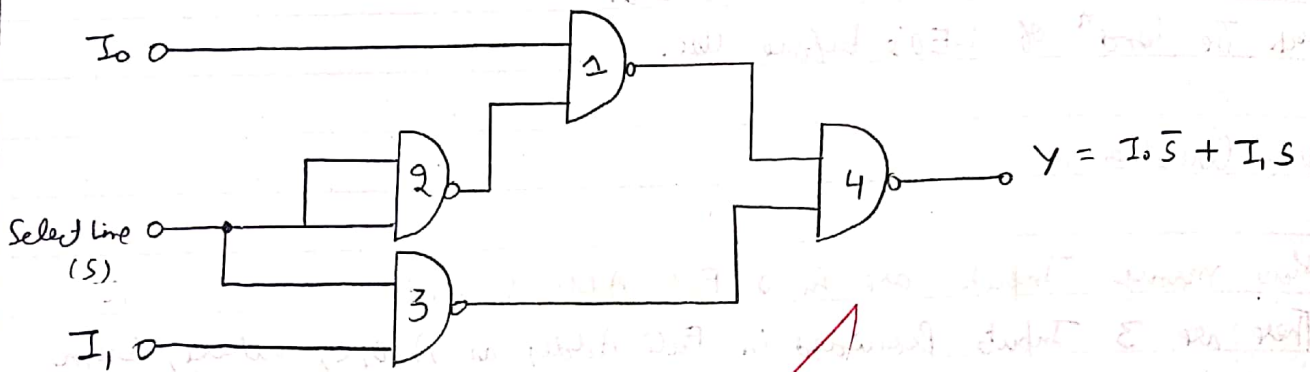
Teacher's Signature: _____

① Realization of 2x1 Mux

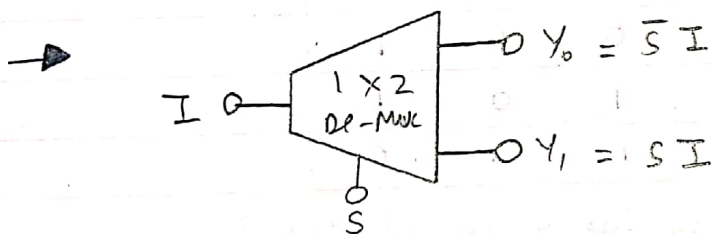


Input	Select Line	Output
I_0, I_1	0	I_0
I_0, I_1	1	I_1

"Truth Table"

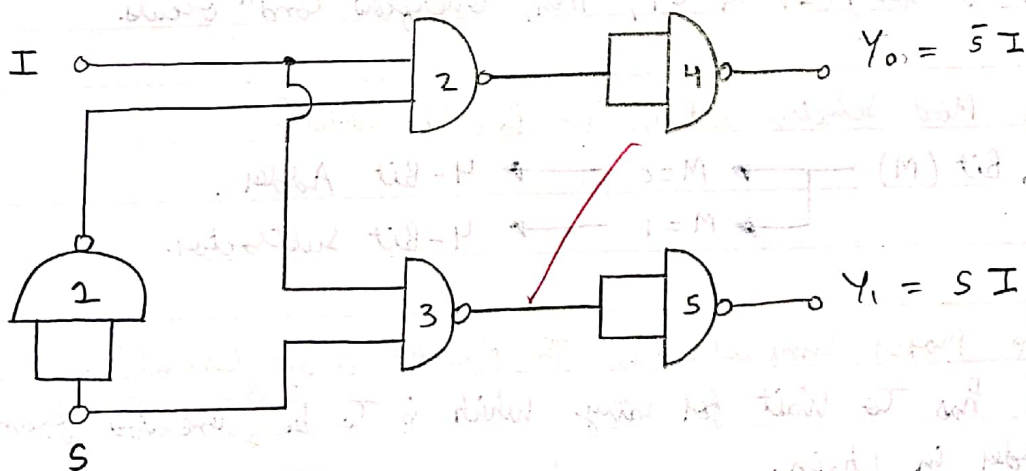


② Realization of 2x1 De-Mux



Input	S	Y_0	Y_1
I	0	I	0
I	1	0	I

"Truth Table"



6. Precautions

- All The Connections must be Tight.
- Always Connect The Ground first and Then The Supply.
- Switch off The Power Supply after The Completion of Experiment.
- Viva - Questions.

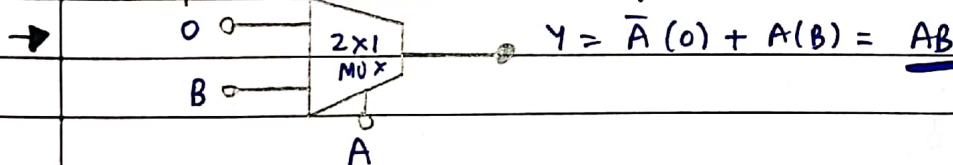
Q → What is Multiplexer?

- Multiplexers are Data Selector which have 2^n Input line, 1 output line with n -select line which decides The Output.

Q → What is The diff. b/w Decoder & De-Multiplexer?

- In Decoder, It is $(n-2^n)$ Decoder with 1 Enable Bit where The output depends on The Combination Given in Input whereas The De-Multiplexer are $(1-2^n)$ with n -select line where The output is been decided by The Combination of Select line.

Q → Implement AND Gate Using Multiplexer?



Q → How many 2x1 Mux are Required To make 8x1 Mux?

- 7 (2x1 Mux) will Be Required.

Teacher's Signature: _____

Experiment - 06.

1. Aim → Design and Implementation of Encoder & Decoder.

2. Apparatus Required

S.NO	Component	Specification	Qty.
1.	3-Input AND Gate	IC-7411	2/2
2.	NOT Gate	IC-7404	1
3.	2 Input OR Gate	IC-7432	2

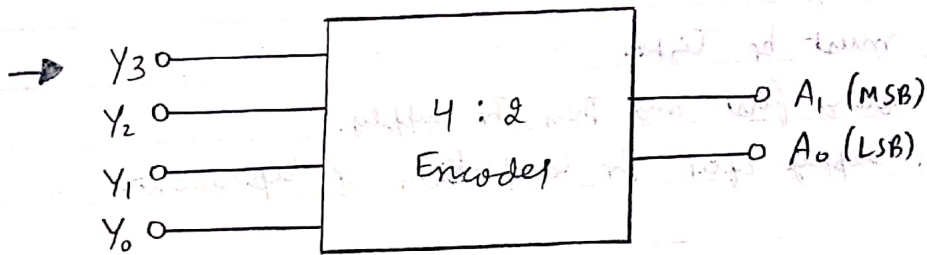
3. Theory

(i) Encoder → The Combinational Circuit that change the Binary Information into N -output lines are called Encoders. The Binary Information is been passed in the form of 2^N Input lines. The Output line define the N -Bit Code for the Binary Information. The Encoder performs the Reverse operation of the Decoder. Only one Input line is been Activated for simplicity and it produces an equivalent N -Bit Binary Information. In 4×2 Encoder, there are total 4 Input Y_3, Y_2, Y_1, Y_0 and 2 Output as A_1 & A_0 . One Input line is set to be True at a time to get Binary Code at output.

(ii) Decoder → The Combinational Circuit that changes the Binary Info. into 2^N Output lines. The Binary Information is been passed in the form of N -Input line. Decoder performs the Reverse operation of Encoder. At a time, 2^N Bit Output Code is equivalent to the Input Binary Code. In 2×4 Decoder, there is total of 3 Input as A_1, A_0 & $E=1$ and it contains 4 Output i.e. Y_3, Y_2, Y_1, Y_0 . For each combination of circuit, when $E=1$, then, one of the Output will be 1.

Teacher's Signature: _____

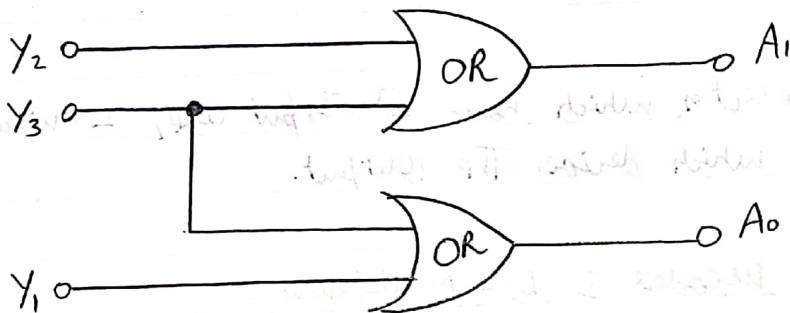
1. Realization of 4x2 Encoder



"Block Diagram of 4x2 Encoder"

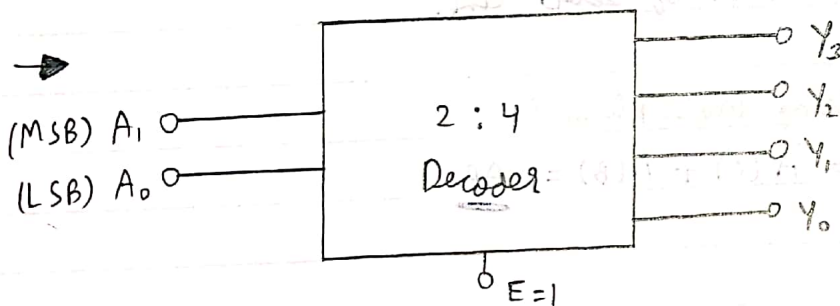
Inputs				Output	
Y_0	Y_1	Y_2	Y_3	A_1	A_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

"Truth Table"



"Circuit Diagram of 4x2 Encoder"

2. Realization of 2x4 Decoder

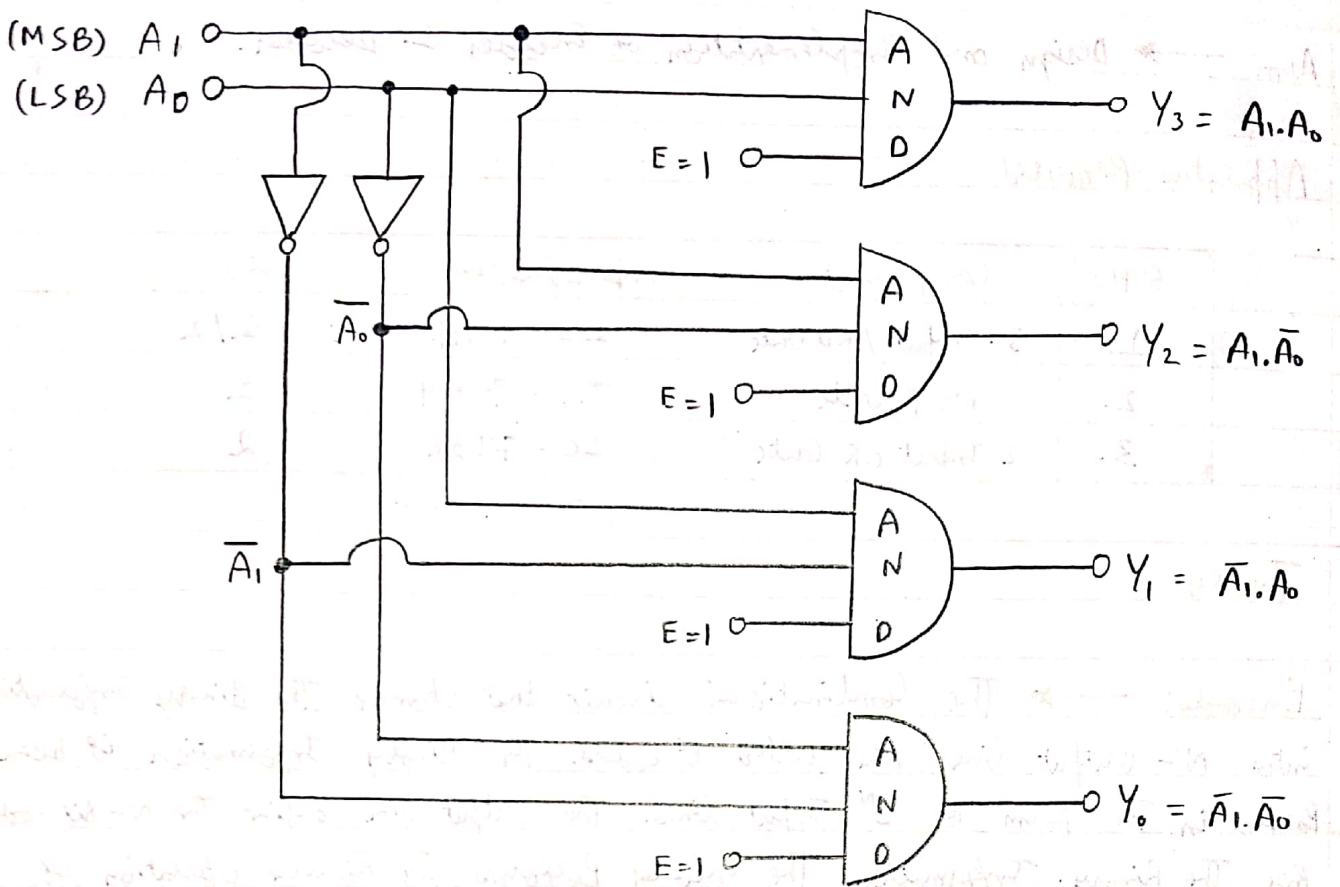


"Block Diagram of 2x4 Decoder"

Input		Outputs			
A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

"Truth Table"

* Circuit Diagram of 2x4 Decoder Circuit



4. Procedure

- Connect The Circuit as shown in The Circuit Diagram.
- Give The Input Value as Specified in Truth Table.
- Switch On The Power Supply.
- Verify The Outputs for Each Input Combination.
- Repeat The Procedure with all Inputs of Truth Table.

5. Result

- Encoder and Decoder have been studied and Their Truth Table have been Verified.

6. Precautions

- All The Connections must be Tight.
- Always Connect The Ground first and Then Supply.
- Switch off The Power Supply after The Completion of Experiment.

→ Viva - Questions.

Q → What are The Application of Decoder?

→ The Application of Decoder are:-

- Decoders are Used for Code Conversions.
- Decoders are used for Memory Systems of Computers.
- Decoders are used for Data Distribution.

"ON NEXT PAGE"
~~~~~

Teacher's Signature: \_\_\_\_\_

Q → What is The Difference b/w Encoder And Decoder?

→ Encoder

Decoder

→ Encoder are  $2^n \times n$  Circuit where They Have  $2^n$  Inputs and  $n$  Output Bits.

→ Decoder are  $n \times 2^n$  Circuit where They Have  $n$  Bit Input &  $2^n$  Bit output.

→ They are Used for Code Conversion of Octal To Binary.

→ They are used for Code Conversion of Binary To Octal.

Q → What are The Application of Encoder?

→ The Application of Encoder are :-

→ Used for Code Conversion of Octal To Binary.

→ Used in Automotive Industry.

→ Used in The Memory Component of CPU.

Q → Define The Boolean Expression of Encoder & Decoder?

→ Encoder →

| $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ | $A_1$ | $A_0$ |
|-------|-------|-------|-------|-------|-------|
| 0     | 0     | 0     | 1     | 0     | 0     |
| 0     | 0     | 1     | 0     | 0     | 1     |
| 0     | 1     | 0     | 0     | 1     | 0     |
| 1     | 0     | 0     | 0     | 1     | 1     |

$$\Rightarrow A_1 = Y_2 + Y_3$$

$$A_0 = Y_3 + Y_1$$

→ Decoder →

| $A_1$ | $A_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
|-------|-------|-------|-------|-------|-------|
| 0     | 0     | 0     | 0     | 0     | 1     |
| 0     | 1     | 0     | 0     | 1     | 0     |
| 1     | 0     | 0     | 1     | 0     | 0     |
| 1     | 1     | 1     | 0     | 0     | 0     |

$$\Rightarrow Y_3 = A_1 A_0$$

$$Y_2 = A_1 \bar{A}_0$$

$$Y_1 = \bar{A}_1 A_0$$

$$Y_0 = \bar{A}_1 \bar{A}_0$$

Teacher's Signature: \_\_\_\_\_



## Experiment - 07.

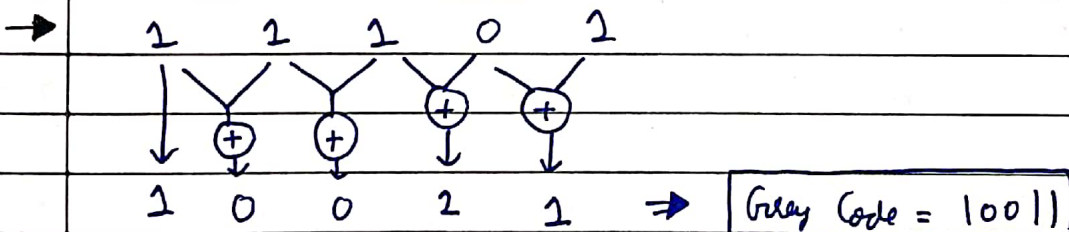
1. Aim  $\rightarrow$  To Realize Grey To Binary and Binary To Grey Converters.

2. Apparatus  $\rightarrow$  Breadboard, IC-7486 (XOR), LEDs, 5V Battery, Wires.

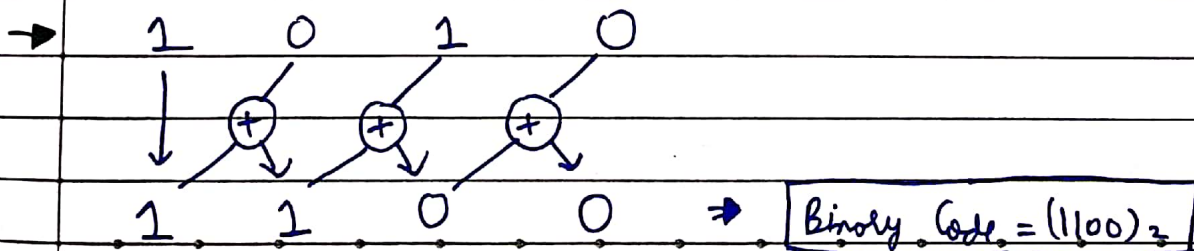
3. Theory

$\rightarrow$  The Grey Code is a Non-Weighted Code. The Successive Grey Code differs in one Bit position which means That It is a Unit Distance Code. It is also The most Popular Cyclic Code. It is Not Suitable for Arithmetic operations. An  $n$ -Bit Grey Code Can be obtained by Reflecting an  $(n-1)$  Bit Code about an Axis after  $2^{n-1}$  rows and Putting the MSB of 0 above The Axis and MSB of 1 below The Axis. The below Solved Example may Useful To Understand how To perform Binary  $\rightarrow$  Grey and Grey  $\rightarrow$  Binary. This Conversion method strongly follows EX-OR Gate operation b/w Binary Bits.

(A) Convert  $(11101)_2$  in Grey Code

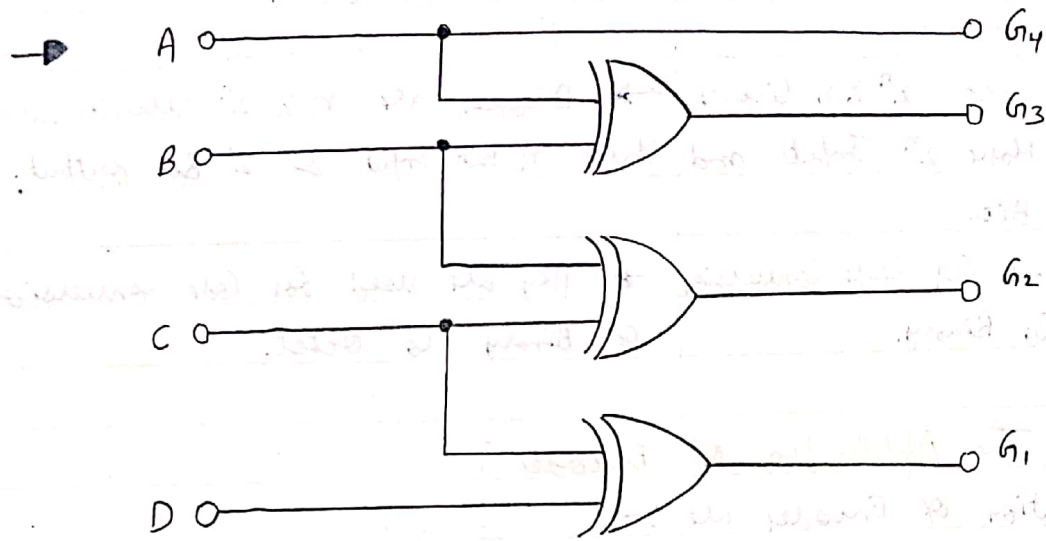


(B) Convert Grey Code 1010 To Binary

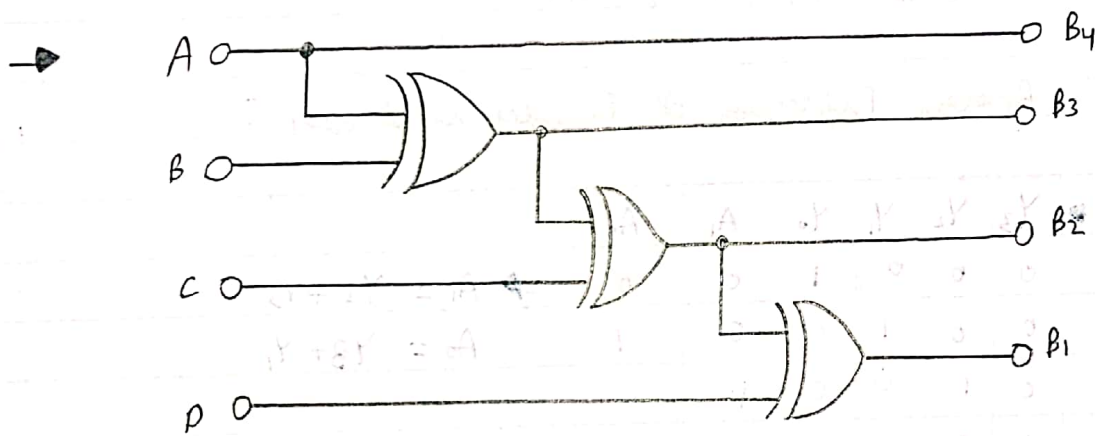


Teacher's Signature: \_\_\_\_\_

## 1. Logic Diagram of Binary To Grey



## 2. Logic Diagram of Grey To Binary



|                              |   |   |   |   |   |
|------------------------------|---|---|---|---|---|
| $A \oplus A = 0$             | 1 | 0 | 0 | 0 | 0 |
| $\bar{A} \oplus A = 1$       | 0 | 1 | 0 | 0 | 1 |
| $A \oplus \bar{A} = 1$       | 0 | 0 | 1 | 0 | 1 |
| $\bar{A} \oplus \bar{A} = 0$ | 0 | 0 | 0 | 1 | 1 |

4. Procedure

- Check all The Components and Their Working.
- Insert The Appropriate IC in The Base.
- Make Connections as Shown in The Circuit Diagram.
- Verify The Results and observe The Output.

5. Result

- Gray → Binary and Binary → Gray has been studied and Verified.

6. Precautions

- All IC's Should be checked before The Start of Experiment.
- All The Connections must be Tight.
- Always Connect Ground first and Then Supply.
- Switch off The Power Supply after Completion of Experiment.

⇒ Viva - Questions.

Q → Why Gray Code is Called as Unit Distance Code?

- Gray Code is also referred as Unit distance Code because it Ensures that Only one bit Changes at a Time as you move from one Value To The next Value. In Gray Code, Adjacent N.O. differ by only one Bit.
- Gray Code is also Called as "Single-bit Change" Code.

"P.T.O"

Teacher's Signature: \_\_\_\_\_



Q → What are The Application of Gray Code in DDCD?

→ The Application of Gray Code are:-

→ Analog - To - Digital Converter (ADC).

→ Error detection and Correction.

→ Binary Coded Decimal (BCD)

→ Simplification using K-Map.

Q → What are Reflected Codes?

→ The Reflected Binary Codes (RBC) also known as Reflected Binary is an Ordering of The Binary Numerical System such That Two Successive Values differ Only by one Bit (Binary digit).

Q → Why Gray Code is used in K-Map instead of Binary?

→ Gray Code Ensures That only one Variable change between The Each Pair of The Adjacent Cells. Each cell of The Completed K-Map Contains an - Binary digit Representing the function's output for The Combination of Input.

Teacher's Signature: \_\_\_\_\_

Experiment - 08.

1. Aim  $\rightarrow$  Construction and Verification of 2-Bit Ripple Counter.

2. Apparatus Required  $\rightarrow$  2 JK-FF (IC-7476), 3 Input NAND Gate and One Switch.

3. Theory

$\rightarrow$  Asynchronous Counter clock pulse is applied only to the initial flip-flop whose value would be considered as LSB instead of the clock pulse. The output of the first flip-flop and so on. Thus, asynchronous counter after the transition of the previous flip-flop transition of the next flip-flop takes place and not at the same time as seen in synchronous counter. Here, flip-flops are in master-slave configuration.

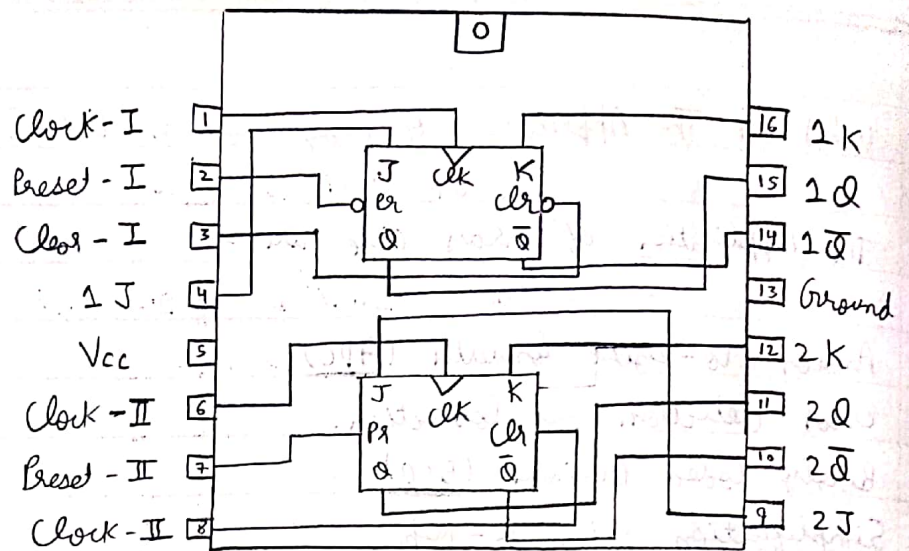
$\rightarrow$  Ripple counter is an asynchronous counter. It got its name because the clock pulse ripples through the circuit. An  $n$ -MOD ripple counter contains  $n$ -number of flip-flops and the circuit can count upto  $2^n$  values before it resets itself to the initial value.

4. Procedure

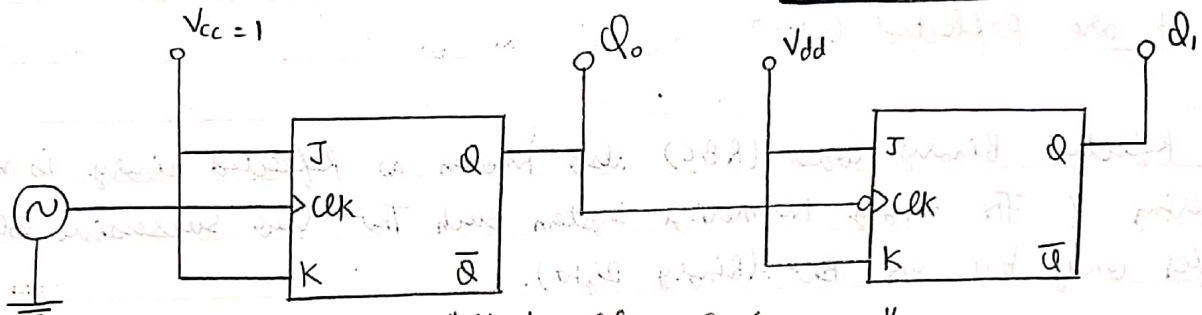
- $\rightarrow$  Connect the circuit as shown.
- $\rightarrow$  Give the input values to the truth table.
- $\rightarrow$  Switch on the power supply.
- $\rightarrow$  Verify the outputs.
- $\rightarrow$  Repeat the process with the truth table.

5. Result  $\rightarrow$  2-Bit Ripple Counter is been Verified.

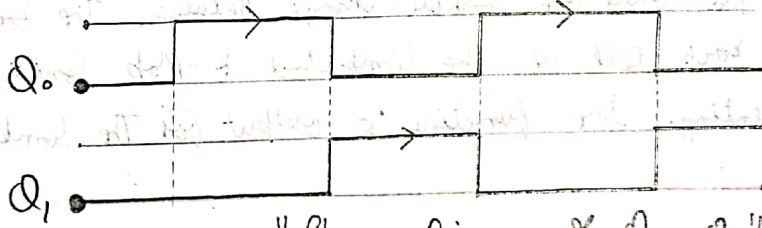
Teacher's Signature: \_\_\_\_\_



"IC-7476"

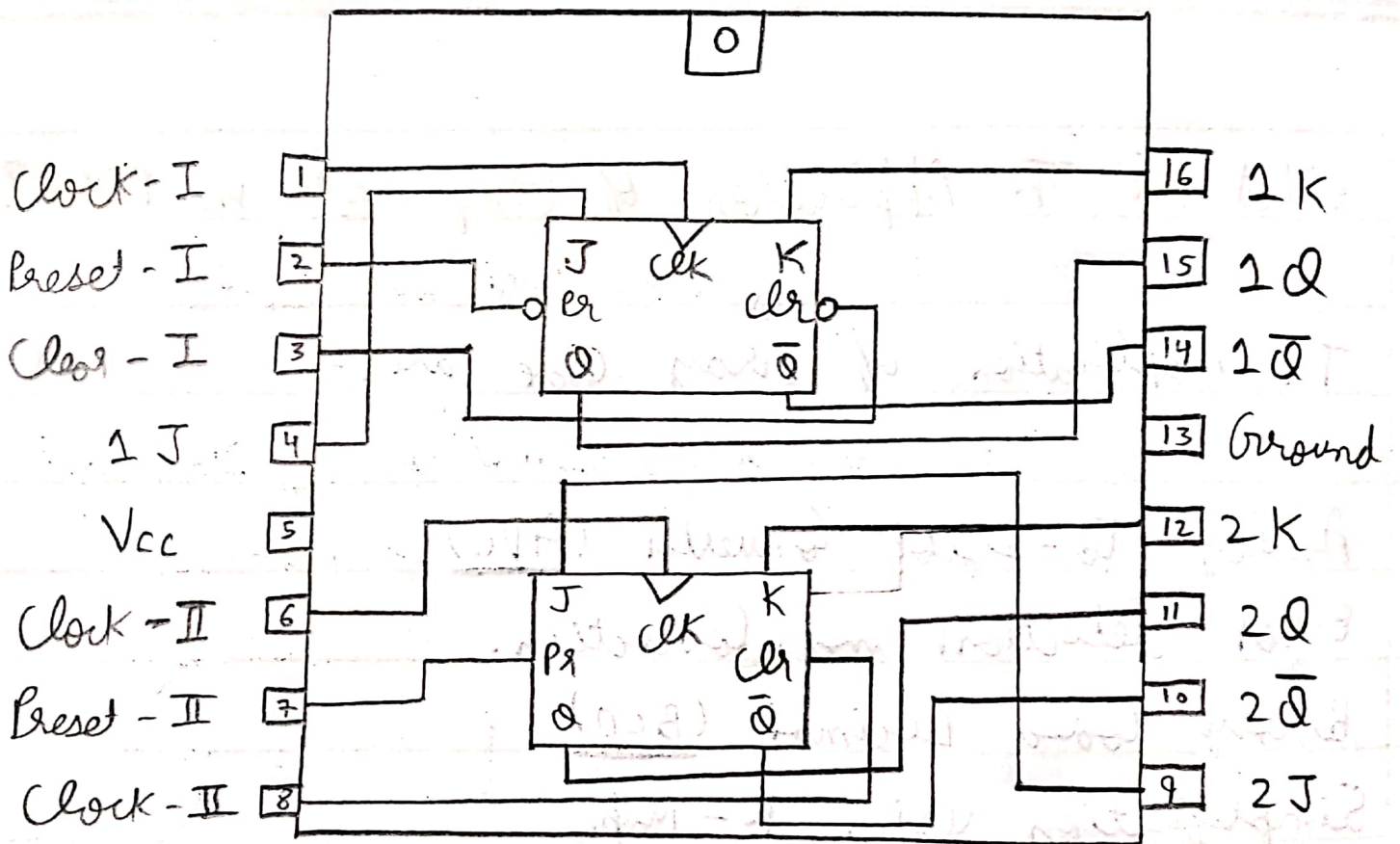


"Master Slave Configuration"



"Phase Diagram of  $Q_0, Q_1$ "





"IC - 7476"

6. Precautions

- All The IC's must be checked before The start of Experiment.
- All The Connections must be Tight.
- Always Connect The Ground first and Then Supply.

⇒ Viva - Questions.

Q → How many Natural States will be In 2-Bit Ripple Counter?

→ 2-Bit Ripple Counter can Count upto 4 States. (00, 01, 10, 11).

Q → How many Different States in an 3-bit Asynchronous Counter?

→ A 3 Bit Ripple Counter will have 8 Different States.

Q → A 5-Bit Asynchronous Binary Counter is made with 5 Flip Flops and Each with a 12 ns Propagation Delay, what is The Total ( $T_{pd}$ )?

→ As,  $n = \text{N.O. of Flip-Flop} = 5$

$T_{pd} = \text{Propagation Delay of FF} = 12 \text{ ns}$

∴ Then,  $(T_{pd})_{\text{net}} = n \times T_{pd}$

$$(T_{pd})_{\text{net}} = 5 \times 12$$

$$(T_{pd})_{\text{net}} = 60 \text{ ns}$$

∴ Thus, Net Propagation Delay will be of 60 ns.

Teacher's Signature: \_\_\_\_\_

|                                                     |          |          |          |
|-----------------------------------------------------|----------|----------|----------|
| <b>Paper Code(s): ECC-253</b>                       | <b>L</b> | <b>P</b> | <b>C</b> |
| <b>Paper: Digital Logic and Computer Design Lab</b> | <b>-</b> | <b>2</b> | <b>1</b> |

**Marking Scheme:**

1. Teachers Continuous Evaluation: 40 marks
2. Term end Theory Examinations: 60 marks

**Instructions:**

1. The course objectives and course outcomes are identical to that of (Digital Logic and Computer Design) as this is the practical component of the corresponding theory paper.
2. The practical list shall be notified by the teacher in the first week of the class commencement under intimation to the office of the Head of Department / Institution in which the paper is being offered from the list of practicals below. Atleast 10 experiments must be performed by the students, they may be asked to do more. Atleast 5 experiments must be from the given list.

1. Design and implementation of adders and subtractors using logic gates.
2. Design and implementation of 4-bit binary adder/subtractor.
3. Design and implementation of multiplexer and demultiplexer.
4. Design and implementation of encoder and decoder.
5. Construction and verification of 4-bit ripple counter and Mod-10/Mod-12 ripple counter.
6. Design and implementation of 3-bit synchronous up/down counter.
7. Design and computer architecture: Design a processor with minimum number of instructions, so that it can do the basic arithmetic and logic operations.
8. Write an assembly language code in GNUsim8085 to implement data transfer instruction.
9. Write an assembly language code in GNUsim8085 to store numbers in reverse order in memory location.
10. Write an assembly language code in GNUsim8085 to implement arithmetic instruction.
11. Write an assembly language code in GNUsim8085 to add two 8 bit numbers.
12. Write an assembly language code in GNUsim8085 to find the factorial of a number.
13. Write an assembly language code in GNUsim8085 to implement logical instructions.
14. Write an assembly language code in GNUsim8085 to implement stack and branch instructions.



Experiment - 09.

Q → Write an Assembly language Code in GNUsim 8085 To Implement Data Transfer Instruction ?

→ LXI H, DF10 H ;  
LXI D, DF70 H ;  
MVI B, 10H ;  
LOOP : MOV A, M ;  
STAX D ;  
INX H ;  
INX D ;  
DCR B ;  
JNZ LOOP ;  
HLT ;

Teacher's Signature: .....

## **Output Of Program-09.**

### Registers

|       |          |
|-------|----------|
| A/PSW | 0x 00 56 |
| BC    | 0x 00 00 |
| DE    | 0x DF 80 |
| HL    | 0x DF 20 |
| SP    | 0x FF FF |
| PC    | 0x 08 11 |

### Flags

|    |                                     |
|----|-------------------------------------|
| Z  | <input checked="" type="checkbox"/> |
| S  | <input type="checkbox"/>            |
| P  | <input checked="" type="checkbox"/> |
| C  | <input type="checkbox"/>            |
| AC | <input checked="" type="checkbox"/> |

Load at 0x0800

▶
◀
⏮
⏭

main.asm

```

1 LXI H, DF10H; Load extended Register Immediately
2 LXI D, DF70H
3 MVI B,10H; Immediately move to B
4 LOOP: MOV A,M
5 STAX D; Store the extended Register
6 INX H; Increase
7 INX D
8 DCR B; Decrease
9 JNZ LOOP; Jump to Loop if no Z flag
10 HLT; Halt the Program

```

### Memory View

0x
DF10

|     | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|-----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| DF0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DF1 | 50 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DF2 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DF3 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DF4 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DF5 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DF6 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DF7 | 50 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DF8 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DF9 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DFA | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DFB | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DFC | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DFD | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DFE | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| DF7 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |

Start Address at 0x

DD00

Experiment - 10

Q → Write an Assembly language Code in GNUsim 8085 To Implement Arithmetic Instructions on The Values?

→ LXI B, 0000;  
MVI C, 0AH;

MOV A, B;

ADD C;

MOV B, A;

MOV A, B;

SUB C;

MOV B, A;

HLT;

Teacher's Signature: \_\_\_\_\_



## Output Of Program-10.

The screenshot displays an 8086 assembly simulator interface. On the left, the **Registers** panel shows the state of various registers: A/PSW (0x0056), BC (0x000A), DE (0x0000), HL (0x0000), SP (0xFFFF), and PC (0x080C). Below this, the **Flags** panel shows Z (checked), S (unchecked), P (checked), C (unchecked), and AC (checked). The central panel displays the assembly code for `main.asm`, with line 1 highlighted: `1 LXI B,0000;Load extended Register B Immediately`. The right panel shows the **Memory View** at address 0x0800, with a table of memory contents. The first byte at address 000 is FF, while all other bytes are 00. At the bottom, the **Start Address** is set to 0.

**Registers**

| Register | Value  |
|----------|--------|
| A/PSW    | 0x0056 |
| BC       | 0x000A |
| DE       | 0x0000 |
| HL       | 0x0000 |
| SP       | 0xFFFF |
| PC       | 0x080C |

**Flags**

| Flag | Status                              |
|------|-------------------------------------|
| Z    | <input checked="" type="checkbox"/> |
| S    | <input type="checkbox"/>            |
| P    | <input checked="" type="checkbox"/> |
| C    | <input type="checkbox"/>            |
| AC   | <input checked="" type="checkbox"/> |

**main.asm**

```
1 LXI B,0000;Load extended Register B Immediately
2 MVI C,0AH;Move Immediately to C
3
4 MOV A,B;Move B to A
5 ADD C; Add C to A
6 MOV B,A;Move A to B
7
8 MOV A,B;Move B to A
9 SUB C;Subtract C from A
10 MOV B,A; Move A to B
11
12 HLT; Halt the Program
```

**Memory View**

| Address | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|---------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 000     | FF | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 001     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 002     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 003     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 004     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 005     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 006     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 007     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 008     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 009     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 00A     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 00B     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 00C     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 00D     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 00E     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 00F     | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |

Start Address at: 0x 0

## Experiment - 11.

~~~~~

Q → Write Assembly language Code in GNU sim 8085 To Add Two 8-Bit Number?

→ LXI H, 3000 H;

MOV B, M;

INX H;

MOV A, M;

ADD B;

MOV B, A;

HLT;

Teacher's Signature: _____

Output Of Program-11.

The screenshot displays a 68000 assembly simulator interface. On the left, the **Registers** panel shows the current state of the processor registers: A/PSW is 0x8082, BC is 0x8000, DE is 0x0000, HL is 0x3001, SP is 0xFFFF, and PC is 0x0809. Below this, the **Flags** panel shows the status of various flags: Z (Zero) is false, S (Sign) is true, P (Parity) is false, C (Carry) is false, and AC (Auxiliary Carry) is false. The central panel shows the assembly code for `main.asm`, which consists of nine instructions: `1 LXI H,3000H; Load Extended Register Immediately`, `2 MOV B,M; Move M to B`, `3 INX H; Increase H`, `4 MOV A, M; Move M to C`, `5` (blank), `6 ADD B; Add B to A`, `7 MOV B,A; Move B to A`, `8` (blank), and `9 HLT`. The right panel, **Memory View**, shows a hexadecimal dump of memory starting at address 0x3000. The first two bytes at address 300 are 50 and 30, while all other bytes are 00. At the bottom right, the **Start Address** is set to 0x2200.

Registers

A/PSW 0x8082

BC 0x8000

DE 0x0000

HL 0x3001

SP 0xFFFF

PC 0x0809

Flags

Z ☐

S ☒

P ☐

C ☐

AC ☐

main.asm

```
1 LXI H,3000H; Load Extended Register Immediately
2 MOV B,M; Move M to B
3 INX H; Increase H
4 MOV A, M; Move M to C
5
6 ADD B; Add B to A
7 MOV B,A; Move B to A
8
9 HLT
```

Load at 0x0800

Memory View

0x 3000

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
300	50	30	00	00	00	00	00	00	00	00	00	00	00	00	00
301	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
302	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
303	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
304	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
305	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
306	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
307	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
308	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
309	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
30A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
30B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
30C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
30D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
30E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
30F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Start Address at: 0x

2200

Experiment - 12.

Q → Write an Assembly language Code in GNU sim 8085 To find factorial of No?

→ LXI SP, 27FFH;

LD A, Var2;

CPI 02H;

JC last;

MVI D, 00H;

MOV E, A;

DEC A;

MOV C, A;

CALL fact;

XCHG;

SHLD Var;

IMP END;

last: LXI H, 0001H;

END: SHLD Var;

HLT;

fact: LXI H, 0000H;

MOV B, C;

back: DAD D;

DEC B;

JNZ back;

Teacher's Signature: _____

Output Of Program-12.

The screenshot displays a debugger window with three main panels: Registers, Assembly Code, and Memory View.

Registers Panel: Shows the state of various registers. A/PSW is 0x7856. BC is 0x0078. DE is 0x0000. HL is 0x0000. SP is 0xFFFF. PC is 0x0812. The Flags section shows Z (checked), S (unchecked), P (checked), C (unchecked), and AC (checked).

Assembly Code Panel: Displays the assembly code for 'main.asm'. The code includes instructions like 'lxi sp,27ffh', 'lda var2', 'cpi 02h', 'jc last', 'mvi d,00h', 'mov e,a', 'dcr a', 'mov c,a', 'call facto', 'xchg', 'shld var', 'jmp end', 'last: lxi h,0001h', 'end: shld var', 'hlt', 'facto: lxi h,0000h', 'mov b,c', 'back: dad d', 'dcr b', and 'jnz back'.

Memory View Panel: Shows a memory dump starting at address 0x270. The dump consists of a grid where each row represents a memory address (from 270 to 27F) and each column represents a byte (from 0 to F). All bytes in the dump are 00.

At the bottom of the Memory View panel, there is a 'Start Address at: 0x' field with a dropdown menu set to '2200' and a slider control.